



## Avaliação 2

JavaScript (Temas 2 a 6) + Ferramentas para Desenvolvimento Web

### Questão 1 – Verdadeiro (V) ou Falso (F) – 0,4

Ferramentas

Analise as afirmações sobre **Ferramentas para Desenvolvimento Web**:

- I. O GitHub Pages exige que o arquivo principal se chame `index.html` e esteja na raiz do projeto.
- II. A extensão Live Preview do VS Code mostra a página em um navegador externo com atualização automática.
- III. Arquivos com extensão `.json` armazenam dados estruturados e não são executados diretamente pelo navegador.

**Assinale a alternativa correta:**

- ☐ a) V – V – F
- ☒ b) V – F – V
- ☐ c) F – V – V
- ☐ d) V – V – V
- ☐ e) F – F – V

**Questão 2 – Verdadeiro (V) ou Falso (F) – 0,4**

JavaScript

Analise as afirmações sobre **Operadores em JavaScript**:

- I. O operador `%` retorna o resto da divisão entre dois números.
- II. O operador `||` (OR) retorna `true` se pelo menos um dos operandos for verdadeiro.
- III. O operador `!` (NOT) inverte o valor booleano do operando.

**Assinale a alternativa correta:**

- ☐ a) V – V – F
- ☐ b) V – F – V
- ☐ c) F – V – V
- ☒ d) V – V – V
- ☐ e) F – F – V

**Questão 3 – Múltipla Escolha – 0,35**

Ferramentas

Sobre o **OneCompiler**, assinale a alternativa **CORRETA**:

- ☐ a) O OneCompiler executa o código JavaScript diretamente no navegador do usuário
- ☐ b) O OneCompiler usa o motor V8 do navegador para executar o código
- ☒ c) O OneCompiler envia o código para servidores remotos que executam com Node.js
- ☐ d) O OneCompiler não funciona com JavaScript, apenas com Python
- ☐ e) O OneCompiler compila o JavaScript para linguagem de máquina antes de executar

**Questão 4 – Múltipla Escolha – 0,35**

JavaScript

Analise o código abaixo:

```
let contador = 0;

while (contador < 4) {
  console.log(contador);
  contador++;
}
```

Qual será a saída exibida no console?

- ☒ a) 0 1 2 3
- ☐ b) 1 2 3 4
- ☐ c) 0 1 2 3 4
- ☐ d) 1 2 3
- ☐ e) Loop infinito

**Questão 5 – Múltipla Escolha – 0,35**

JavaScript

Qual método de array em JavaScript é utilizado para **remover e/ou substituir elementos** em uma posição específica?

- ☐ a) push()
- ☐ b) pop()
- ☐ c) shift()
- ☒ d) splice()
- ☐ e) unshift()

**Questão 6 – Múltipla Escolha – 0,35**

JavaScript

Qual estrutura condicional em JavaScript é mais adequada para verificar o dia da semana (1 a 7) e exibir seu nome correspondente?

- ☐ a) if simples
- ☐ b) if/else composta
- ☒ c) switch
- ☐ d) for
- ☐ e) while

**Questão 7 – Questão Aberta – 0,45**

Dissertativa

**Explique detalhadamente** como funciona o **GitHub Pages** e quais são os requisitos para publicar um site.

Inclua em sua resposta: o que é o serviço, qual arquivo é obrigatório, onde ele deve estar, qual branch é usada e qual é o formato da URL gerada.

O Github Pages é um serviço que permite a hospedagem de sites estáticos direto de repositórios. O mesmo exige que o HTML esteja na forma de index.html e esteja na raiz do repositório. Após isso, a branch do repositório deve estar definida como "main" e o Github irá gerar um URL

**Questão 8 – Questão Aberta – 0,45**

Dissertativa

**Explique detalhadamente** as limitações de segurança do JavaScript quando executado no navegador.

Inclua em sua resposta: o que o JavaScript NÃO pode fazer no navegador, o que ele PODE fazer e por que existem essas restrições.

O JavaScript quando executado no navegador mantém boa parte suas funções normais como a manipulação do DOM, porém há muitas restrições em relação ao carregamento de arquivos locais, execução de comandos no

**Questão 9 – Questão Aberta – 0,45**

Dissertativa

**Explique detalhadamente** a diferença entre os operadores `==` e `===` em JavaScript.

Inclua em sua resposta: o que cada um compara, como funcionam as conversões automáticas de tipo, exemplos práticos e qual é a boa prática recomendada.

No JavaScript, o operador `==` apenas compara o valor e faz conversões entre numbers e strings, enquanto `===` compara valor e tipo e não faz conversões.

**Questão 10 – Questão Aberta – 0,45**

Dissertativa

**Explique detalhadamente** a diferença entre os laços `for` e `while` em JavaScript.

Inclua em sua resposta: a sintaxe de cada um, em qual situação cada é mais adequado, e exemplos práticos de uso.

```
let i = 0;
while (i < 10) {
  console.log(i);
  i++;
}
```

**Gabarito Completo com Explicações****Questão 1 – Verdadeiro (V) ou Falso (F)**

✓ Resposta correta: **b) V – F – V**

**Explicação detalhada:**

- **I. VERDADEIRO** — O GitHub Pages **exige** que o arquivo principal seja `index.html` e esteja na **raiz** do projeto. É obrigatório.
- **II. FALSO** — O **Live Preview** mostra a página **dentro do VS Code**. Quem mostra em navegador externo com atualização automática é o **Live Server**.
- **III. VERDADEIRO** — Arquivos `.json` armazenam dados estruturados e **NÃO** são executados pelo navegador — são apenas lidos.

**Questão 2 – Verdadeiro (V) ou Falso (F)**

✓ Resposta correta: **d) V – V – V**

**Explicação detalhada:**

- **I. VERDADEIRO** — O operador % (módulo) retorna o **resto da divisão** entre dois números. Ex:  $10 \% 3 = 1$ .
- **II. VERDADEIRO** — O operador || (OR) retorna true se **pelo menos um** dos operandos for verdadeiro. Ex: `true || false = true`.
- **III. VERDADEIRO** — O operador ! (NOT) **inverte** o valor booleano. Ex: `!true = false`.

**Questão 3 – Múltipla Escolha**

✓ Resposta correta: **c) O OneCompiler envia o código para servidores remotos que executam com Node.js**

**Explicação detalhada:**

**Como funciona o OneCompiler:**

- ✗ **NÃO** executa no navegador (alternativa a está errada)
- ✗ **NÃO** usa o motor V8 do navegador (alternativa b está errada)
- ✓ Envia o código para **servidores remotos** que executam com **Node.js**
- ✗ **NÃO** compila para linguagem de máquina (alternativa e está errada)

💡 **IMPORTANTE:** O OneCompiler é um ambiente **server-side** (Node.js), não client-side.

**Questão 4 – Múltipla Escolha**

✓ Resposta correta: **a) 0 1 2 3**

**Explicação detalhada:**

**Analisando o código:**

- `contador = 0` → condição `0 < 4` é **verdadeira** → imprime 0, incrementa para 1
- `contador = 1` → condição `1 < 4` é **verdadeira** → imprime 1, incrementa para 2
- `contador = 2` → condição `2 < 4` é **verdadeira** → imprime 2, incrementa para 3
- `contador = 3` → condição `3 < 4` é **verdadeira** → imprime 3, incrementa para 4
- `contador = 4` → condição `4 < 4` é **falsa** → o loop termina

**Saída:** 0 1 2 3

**Questão 5 – Múltipla Escolha**

✓ Resposta correta: **d) splice()**

**Explicação detalhada:****Métodos de array:**

- `push()` → adiciona ao **final**
- `pop()` → remove do **final**
- `unshift()` → adiciona ao **início**
- `shift()` → remove do **início**
- `splice()` → remove e/ou substitui em **qualquer posição**

**Sintaxe do splice:**

```
array.splice(índice, quantidade, ...elementosParaAdicionar)
```

**Exemplo:**

```
let cores = ["Vermelho", "Verde", "Azul"];  
cores.splice(1, 1, "Amarelo");  
// ["Vermelho", "Amarelo", "Azul"]
```

**Questão 6 – Múltipla Escolha**

✅ Resposta correta: **c) switch**

**Explicação detalhada:****Por que switch é mais adequado?**

- O switch é ideal para comparar uma expressão com **múltiplos valores fixos**
- Para dias da semana (1 a 7), temos 7 valores fixos para comparar
- O código fica **mais legível** e **mais organizado** que vários if/else

**Exemplo com switch:**

```
switch (dia) {  
  case 1: console.log("Domingo"); break;  
  case 2: console.log("Segunda"); break;  
  // ...  
  default: console.log("Dia inválido");  
}
```

💡 DICA: Use switch para valores fixos e if/else para condições complexas (com >, <, etc.).

**Questão 7 – Questão Aberta**

📝 **Sugestão de Resposta**

**O que é GitHub Pages?**

O GitHub Pages é um serviço **gratuito** da GitHub para hospedar **sites estáticos** diretamente de um repositório.

**Requisitos para publicar:**

- ☒ O arquivo principal **DEVE** se chamar `index.html`
- ☒ O `index.html` deve estar na **raiz** do projeto (pasta principal)
- ☒ O repositório deve ser **público** (na versão gratuita)
- ☒ A branch usada geralmente é a `main` ou `master`

#### URL gerada:

`https://nome-do-usuario.github.io/nome-do-repositorio/`

#### Estrutura recomendada:

```
meu-projeto/  
├── index.html ← OBRIGATÓRIO  
├── style.css  
├── script.js  
├── images/  
└── pages/
```

### Questão 8 – Questão Aberta

#### Sugestão de Resposta

#### Limitações de segurança do JavaScript no navegador:

##### O que JavaScript **NÃO** pode fazer no navegador:

- ☒ **Acessar o sistema de arquivos** — não pode ler ou escrever arquivos no computador do usuário
- ☒ **Acessar o sistema operacional** — não pode executar comandos do sistema
- ☒ **Acessar banco de dados diretamente** — precisa de uma API/servidor intermediário
- ☒ **Fazer requisições para outros domínios** — bloqueado pela política CORS

##### O que JavaScript **PODE** fazer no navegador:

- ☒ Manipular o **DOM** (HTML e CSS) dinamicamente
- ☒ Responder a **eventos** do usuário (cliques, teclas, etc.)
- ☒ Fazer **requisições HTTP** (AJAX, fetch) para o mesmo domínio
- ☒ Validar **formulários** antes do envio
- ☒ Criar **animações** e interações na página
- ☒ Armazenar dados no **localStorage** (pequena quantidade)

#### Por que existem essas restrições?

- **Segurança do usuário:** proteger dados pessoais e evitar ataques maliciosos
- **Privacidade:** impedir que sites acessem informações sensíveis
- **Isolamento:** cada site executa em um "sandbox" separado



## Questão 9 – Questão Aberta

### Sugestão de Resposta

#### Diferença entre == e ===:

##### == (Igualdade - comparação solta):

- Compara apenas o **valor**
- **Converte tipos automaticamente** (coerção de tipos)
- Ex: `10 == "10" → true` (converte string para número)
- Ex: `0 == false → true` (converte false para 0)
- Ex: `null == undefined → true`

##### === (Identidade - comparação estrita):

- Compara **valor E tipo**
- **Não converte tipos**
- Ex: `10 === "10" → false` (tipos diferentes)
- Ex: `0 === false → false` (number vs boolean)
- Ex: `null === undefined → false`

#### Boa prática recomendada:

-  Use `===` sempre que possível
-  **Evite** `==` para evitar resultados inesperados
- O `===` torna o código mais **previsível** e **seguro**

## Questão 10 – Questão Aberta

### Sugestão de Resposta

#### Diferença entre for e while:

##### for (laço contado):

- Usado quando **sabemos o número exato de repetições**
- Possui **controle completo** (inicialização, condição, incremento)
- **Sintaxe:** `for (inicialização; condição; incremento) { ... }`

```
for (let i = 0; i < 10; i++) {  
  console.log(i); // 0, 1, 2, ..., 9  
}
```

##### while (laço condicional):

- Usado quando a repetição **depende de uma condição**
- Não tem controle de contador embutido
- **Sintaxe:** `while (condição) { ... }`

```
let resposta = "";  
while (resposta !== "sair") {
```

```
resposta = prompt("Digite algo:");  
}
```

**Quando usar cada um:**

- **for:** Percorrer arrays, contagens definidas, tabuadas
- **while:** Ler dados até condição, aguardar entrada, validações

**Exemplo prático de cada:**

```
// FOR - percorrer um array  
let frutas = ["Maçã", "Banana", "Laranja"];  
for (let i = 0; i < frutas.length; i++) {  
  console.log(frutas[i]);  
}  
  
// WHILE - aguardar resposta do usuário  
let continuar = true;  
while (continuar) {  
  let resposta = prompt("Continuar? (s/n)");  
  if (resposta === "n") continuar = false;  
}
```